POLITECHNIKA RZESZOWSKA
im. IGNACEGO ŁUKASIEWICZA

KATEDRA INFORMATYKI I AUTOMATYKI POLITECHNIKI RZESZOWSKIEJ

POLITECHNIKA RZESZOWSKA
KATEDRA INFORMATYKI I AUTOMATYKI
DR INŻ. MATEUSZ POMIANEK

ĆWICZENIE LABORATORYJNE NR 5

Hilt, Testy jednostkowe i instrumentacyjne

Programowanie Aplikacji Mobilnych (Android / Kotlin / Jetpack Compose)

Projekty: TaskApp (Hilt) + PokeApp (testy)

Wymagania: ukończone Lab 1–4 (Room, ViewModel, Navigation, Retrofit, Coroutines)

Spis treści

1. Czym jest Dependency Injection? 1
2. Hilt: konfiguracja projektu 2
3. Moduły Hilt; jak dostarczać zależności 4
4. Zakresy (Scopes); cykl życia zależności 5
5. Filozofia testowania i piramida testów 7
6. Testy jednostkowe - JUnit 5 + MockK 8
7. Testy integracyjne - Room InMemory 10
8. Testy instrumentowane - Compose UI Testing 11
9. Hilt w testach - podmienianie zależności 13
10. Testowanie Coroutines: TestDispatcher i Turbine 14
11. Strategia testowania, czyli co testować w projekcie 16
12. Zadania do wykonania 16
13. Kryteria oceniania 17
14. Najczęstsze błędy i ich rozwiązania 18

1. Czym jest Dependency Injection?

Zanim poznamy Hilt, musimy dokładnie zrozumieć problem, który rozwiązuje. Dependency Injection (DI) to jeden z tych wzorców projektowych, który łatwo zrozumieć po zobaczeniu kodu bez niego, jest po prostu nieczytelny i kruchy.

1.1. Świat bez DI - problem twardych zależności

Wyobraź sobie, że piszesz ViewModel, który potrzebuje Repository, które potrzebuje DAO i ApiService. Bez DI każda klasa tworzy swoje zależności samodzielnie:

Anty-wzorzec: twarde zależności

```
// BEZ DI - jak NIE należy robić:
class TaskViewModel : ViewModel() {
  // ViewModel sam tworzy Database - hardcoded 'new'
  private val db = Room.databaseBuilder(        // Wymaga Context!
    MyApp.instance,                             // Singleton aplikacji - anty-wzorzec
```

```
    TaskDatabase::class.java, "tasks.db"
  ).build()
  private val dao        = db.taskDao()         // Hardcoded zależność
  private val apiService = RetrofitClient.api   // Singleton - nie można zamockować!
  private val repository = TaskRepository(dao, apiService)

  // Problem 1: W teście nie można podmienić prawdziwej bazy na fake
  // Problem 2: Zmiana implementacji bazy = zmiana w ViewModelu
  // Problem 3: ViewModel tworzy Room - narusza Single Responsibility
}
```

Analogia: budowa domu i podwykonawcy

Wyobraź sobie budowę domu. Architekt (ViewModel) potrzebuje elektryka, hydraulika i murarza.

TWARDY ZALEŻNOŚĆ (zły sposób): Architekt sam szuka elektryka, dzwoniąc do konkretnego Pana Jana. Jeśli Pan Jan jest chory, budowa stoi. Jeśli chcemy zmienić elektryka to musimy zmienić cały projekt.

DEPENDENCY INJECTION (dobry sposób): Architekt mówi tylko 'potrzebuję elektryka'.
KIEROWNIK BUDOWY (Hilt) dostarcza odpowiedniego fachowca. Może to być Pan Jan, albo Pani Anna, albo na czas testów - aktor który udaje elektryka (mock). Architekt nie wie i nie musi wiedzieć.

Korzyści: łatwa wymiana implementacji, testowalność, single responsibility, brak cykli zależności.

1.2. Trzy sposoby wstrzykiwania

Typ wstrzykiwania	Opis i kiedy używać
`Constructor Injection (preferowany)`	Zależności przekazywane przez konstruktor. Najbardziej czytelny, obowiązkowy dla klas nie-Android (Repository, UseCase, Mapper). Hilt automatycznie dostarcza argumenty.
`Field Injection (@Inject lateinit var)`	Dla klas tworzone przez Android (Activity, Fragment, ViewModel). Hilt wstrzykuje pola oznaczone @Inject automatycznie po @AndroidEntryPoint.
`Method Injection (@Inject fun)`	Rzadko używany. Wywoływany po constructor injection. Przydatny gdy zależność potrzebna dopiero po częściowej inicjalizacji.

Dlaczego właśnie Hilt, a nie ręczne DI lub pure Dagger?

RECZNE DI: Skaluje się do około 10 klas. Przy 50+ klasach 'fabryki zależności' stają się koszmarem - każda zmiana to lawina editów.

PURE DAGGER 2: Potężny, ale wymaga ręcznego pisania Componentów, Moduleów i Subcomponentów. Krzywa uczenia się jest bardzo stroma. Tysiące linii boilerplate kodu.

HILT = Dagger 2 + gotowe komponenty dla Android + zero boilerplate komponentów.
Hilt wie o cyklu życia Activity, Fragment, ViewModel, Service - sam tworzy odpowiednie zakresy.
Mniej kodu do napisania, pełna moc Dagger 2 'pod spodem'.

2. Hilt: konfiguracja projektu

Hilt wymaga konfiguracji na trzech poziomach: wtyczka Gradle (procesor adnotacji), zależności biblioteki, oraz adnotacja @HiltAndroidApp na klasie Application. Bez któregokolwiek z tych elementów Hilt nie działa.

2.1. Wtyczka i zależności

gradle/libs.versions.toml

```
# gradle/libs.versions.toml
[versions]
hilt            = "2.51.1"    # Hilt - zawsze użyj tej samej wersji dla wszystkich artefaktów!
ksp             = "2.0.21-1.0.25"   # KSP - Kotlin Symbol Processing (szybszy niż kapt)

[libraries]
hilt-android    = { module = "com.google.dagger:hilt-android",           version.ref = "hilt" }
hilt-compiler   = { module = "com.google.dagger:hilt-compiler",          version.ref = "hilt" }
hilt-nav-compose= { module = "androidx.hilt:hilt-navigation-compose",    version = "1.2.0" }
hilt-testing    = { module = "com.google.dagger:hilt-android-testing",   version.ref = "hilt" }

[plugins]
hilt            = { id = "com.google.dagger.hilt.android", version.ref = "hilt" }
ksp             = { id = "com.google.devtools.ksp",         version.ref = "ksp" }
```

Konfiguracja build.gradle.kts

```
// build.gradle.kts (project level)
plugins {
  alias(libs.plugins.hilt)  apply false  // Deklaruj, nie stosuj na poziomie projektu
  alias(libs.plugins.ksp)   apply false
}

// app/build.gradle.kts
plugins {
  id("com.android.application")
  id("org.jetbrains.kotlin.android")
  alias(libs.plugins.hilt)   // Wtyczka Hilt - musi być przed dependencies!
  alias(libs.plugins.ksp)    // KSP generuje kod w czasie kompilacji
}

dependencies {
  implementation(libs.hilt.android)         // Runtime Hilt
  ksp(libs.hilt.compiler)                   // Procesor adnotacji - generuje kod DI
  implementation(libs.hilt.nav.compose)      // hiltViewModel() w Compose

  // Dla testów:
  testImplementation(libs.hilt.testing)      // HiltAndroidRule w JVM testach
  androidTestImplementation(libs.hilt.testing)  // HiltAndroidRule w Instrumented testach
  kspTest(libs.hilt.compiler)               // Generowanie kodu DI dla testów
  kspAndroidTest(libs.hilt.compiler)         // Generowanie kodu DI dla Android testów
}
```

⚠ Wersja hilt-compiler MUSI być identyczna z hilt-android!

Hilt używa procesora adnotacji (KSP) do generowania kodu w czasie kompilacji.
Jeśli wersja kompilatora różni się od wersji runtime, Hilt wygeneruje niepoprawny kod.

BŁĄD: hilt-android = 2.51.1 + hilt-compiler = 2.48 → błąd kompilacji, trudny do zdebugowania.
POPRAWNIE: Obie zależności wskazują na version.ref = "hilt" w libs.versions.toml.

Ten sam problem dotyczy par: room-runtime + room-compiler, lifecycle-viewmodel + lifecycle-compiler.
Zawsze użyj jednej zmiennej wersji dla wszystkich artefaktów tej samej biblioteki.

2.2. Klasa Application z @HiltAndroidApp

MyApp.kt - punkt wejścia Hilt	Wyjaśnienie
`@HiltAndroidApp`	@HiltAndroidApp uruchamia generator kodu Hilt dla całej aplikacji
`class MyApp : Application() {`	Dziedziczy po Application - nadpisuje klasa bazowa Androida
`  // Hilt automatycznie tworzy`	Nie musisz nic pisac! Hilt generuje komponent App w tle.
`  // AppComponent pod spodem.`	
`}`	
`// W AndroidManifest.xml:`	Musisz zarejestrować klase Application w Manifeście!
`<application`	
`  android:name=".MyApp"`	Bez tej linii Android użyje domyślnej klasy Application
`  ...>`	Brak .MyApp = Hilt nie inicjalizuje się = crash przy @Inject

3. Moduły Hilt; jak dostarczać zależności

Hilt musi wiedzieć, jak stworzyć każdą zależność. Jeśli klasa ma konstruktor oznaczony @Inject - Hilt poradzi sobie sam. Ale co jeśli musimy dostarczyć interfejs, lub obiekt tworzony przez builder (jak Retrofit lub Room)? Tu właśnie potrzebujemy Modułów.

3.1. Kiedy potrzebny jest @Module?

Sytuacja	Dlaczego potrzebny moduł?
`Interfejs (np. PokemonRepository)`	Hilt nie wie którą implementację wybrać. @Binds wskazuje konkretną klasę.
`Zewnętrzna biblioteka (Retrofit, Room, Gson)`	Nie możesz dodać @Inject do klasy zewnętrznej. @Provides opis opisuje jak ją stworzyć.
`Skomplikowana inicjalizacja (Builder pattern)`	Tworzenie przez builder (Retrofit.Builder()) wymaga kodu - Hilt sam tego nie ogarnie.
`Klasa tworzona fabryką (OkHttpClient.Builder)`	Trzeba skonfigurować timeouty, interceptory - @Provides to miejsce na tę logikę.

3.2. @Provides - jak dostarczyć zewnętrzne klasy

NetworkModule.kt: @Provides dla Retrofit i OkHttp	Wyjaśnienie
`@Module`	@Module informuje Hilt, że ta klasa zawiera przepisy na tworzenie obiektów
`@InstallIn(SingletonComponent::class)`	Określa zakres: Singleton = jeden egzemplarz na całą aplikację
`object NetworkModule {`	object (nie class) - funkcje @Provides mogą być statyczne, to szybsze
`  @Provides`	@Provides = ta funkcja tworzy obiekt dla Hilt
`  @Singleton`	@Singleton = stwórz raz, reedytuj ten sam egzemplarz wszyscie
`  fun provideOkHttpClient(): OkHttpClient {`	Hilt wywoła tę funkcję gdy ktoś poprosi o OkHttpClient
`    return OkHttpClient.Builder()`	Builder pattern - nie można tego zrobić przez @Inject
`      .connectTimeout(10, TimeUnit.SECONDS)`	Konfiguracja timeoutu
`      .build()`	
`  }`	
`  @Provides @Singleton`	Dwie adnotacje na jednej linii - poprawna składnia

NetworkModule.kt: @Provides dla Retrofit i OkHttp	Wyjaśnienie
`fun provideRetrofit(`	Hilt widzi OkHttpClient w parametrze - wstrzykuje automatycznie!
`  client: OkHttpClient  // <- wstrzyknij!`	Hilt użyje provideOkHttpClient() aby dać ten parametr
`): PokemonApiService {`	Zwracamy interfejs - Retrofit wygeneruje implementację
`  return Retrofit.Builder()`	
`  .baseUrl("https://pokeapi.co/api/v2/")`	
`  .client(client)`	Przekazujemy wstrzyknięty klient
`  .addConverterFactory(GsonConverterFactory.create())`	
`  .build()`	
`  .create(PokemonApiService::class.java)`	Dynamic Proxy - implementacja interfejsu
`}`	
`}`	

3.3. @Binds - jak powiązać interfejs z implementacją

RepositoryModule.kt: @Binds dla interfejsu	Wyjaśnienie
`@Module`	
`@InstallIn(SingletonComponent::class)`	
`abstract class RepositoryModule {`	MUSI być abstract class (nie object) gdy używasz @Binds
`  @Binds`	@Binds jest szybszy od @Provides - nie tworzy klasy pośredniej
`  @Singleton`	Jeden egzemplarz repository na app
`  abstract fun bindPokemonRepository(`	@Binds: Hilt widzi że impl = PokemonRepositoryImpl
`  impl: PokemonRepositoryImpl  // <- konkretna klasa`	PokemonRepositoryImpl musi mieć @Inject constructor
`  ): PokemonRepository  // <- interfejs`	Gdy ktoś poprosi o PokemonRepository - dostanie impl
`}`	

🔍 @Provides vs @Binds – którego i kiedy?

@BINDS - gdy masz interfejs + klasę z @Inject constructor. Tylko mapowanie interfejs→implementacja. Hilt generuje prostszy, szybszy kod. Wymaga abstract class (nie object).
Przykład: interface PokemonRepository ← @Binds - PokemonRepositoryImpl

@PROVIDES - gdy musisz wykonać kod tworzenia obiektu (Builder pattern, fabryka, external lib). Funkcja może być nieabstrakcyjna, może być w object (statyczna = szybsza).
Przykład: OkHttpClient.Builder() + timeout + interceptory = potrzebujesz kodu ← użij @Provides.

ZASADA: preferuj @Binds gdy się da, użyj @Provides gdy musisz wykonać kod tworzenia.

4. Zakresy (Scopes): cykl życia zależności

Zakres (scope) określa, jak długo istnieje wstrzyknięty obiekt. To kluczowe pojęcie: zbędne singletonów prowadzi do wycieków pamięci, a zbyt krótki czas życia do wydajnościowych problemów.

🍽 Analogia: okresy ważności biletów

Wyobraź sobie różne rodzaje biletów:

@Singleton = Karta stałego klienta - ważna przez cały czas korzystania z usługi (life of app). Jeden egzemplarz. Wszystkie miejsca dostają tę samą kartę. Przykłady: Database, Retrofit.

@ActivityRetainedScoped = Bilet tygodniowy - przetrwa obrót ekranu (ViewModel jest ActivityRetained). ViewModel Hilt automatycznie używa tego zakresu.

@ViewModelScoped = Bilet na jedną jazdę w obie strony - żyje dopiero do zniszczenia ViewModela. Używaj dla UseCase i helperów, które powinny być unikalne dla każdego ViewModela.

@ActivityScoped = Bilet dzienny - ważny tylko w obrębie jednej Activity (nie przetrwa rotacji!).

4.1. Tabela zakresów Hilt

Adnotacja zakresu	Komponent / Czas życia / Zastosowanie
`@Singleton`	SingletonComponent. Żyje przez całą aplikację. Użyj dla: Database, Retrofit, OkHttpClient, Repository.
`@ActivityRetainedScoped`	ActivityRetainedComponent. Przetrwa rotację ekranu (tak jak ViewModel). Użyj dla: ViewModel-level helperów.
`@ViewModelScoped`	ViewModelComponent. Żyje przez czas życia konkretnego ViewModela. Użyj dla: UseCase związanych z jednym VM.
`@ActivityScoped`	ActivityComponent. Żyje wraz z Activity - umiera przy rotacji. Użyj dla: helperów UI specyficznych dla Activity.
`@FragmentScoped`	FragmentComponent. Żyje wraz z Fragmentem. Rzadko potrzebny w Compose (brak Fragmentów).
`(brak adnotacji)`	Każde wstrzyknięcie tworzy NOWY egzemplarz. Użyj dla: lekkich, bezstanowych helperów.

@Singleton w ViewModelu to wyciek pamięci!

Jeśli wstrzykniesz @Singleton obiekt do Activity (nie przez ViewModel), ten singleton będzie trzymać referencję do Activity - nawet po jej zniszczeniu.

Przykład: @Singleton class UserPreferences(@Inject val context: Context) - Jeśli context to ActivityContext (nie ApplicationContext), masz wyciek pamięci.

ZASADA: Obiekty @Singleton mogą trzymać jedynie ApplicationContext.
Użyj @ApplicationContext zamiast Context tam gdzie to konieczne.

4.2. Hilt i ViewModel: @HiltViewModel

TaskViewModel.kt z Hilt	Wyjaśnienie
`@HiltViewModel`	@HiltViewModel rejestruje VM w systemie Hilt - Hilt zarządza tworzeniem
`class TaskViewModel @Inject constructor(`	@Inject constructor = tutaj Hilt wstrzykuje zależności
`  private val repository: TaskRepository,`	Hilt dostarczy implementację (z @Binds lub @Provides)
`  private val savedStateHandle: SavedStateHandle`	SavedStateHandle - Hilt wie jak to dostarczyć automatycznie!
`) : ViewModel() {`	
`  val tasks = repository.getAllTasks()`	Korzystamy z wstrzykniętego repozytorium
`}`	
`// W Compose UI - dostęp do HiltViewModel:`	
`@Composable fun TaskScreen(`	

TaskViewModel.kt z Hilt	Wyjaśnienie
`viewModel: TaskViewModel = hiltViewModel()`	hiltViewModel() - z libs.hilt.nav.compose
`) {`	hiltViewModel() = Hilt tworzy VM lub zwraca istniejący z zakresu
`val tasks by viewModel.tasks.collectAsStateWithLifecycle()`	
`}`	

5. Filozofia testowania i piramida testów

Testy to nie opcja - to gwarancja, że refaktoryzacja nie psuje istniejących funkcjonalności, a nowy kod robi to co powinien. Dobry projekt testowy opiera się na 'piramidzie testów'.

🍽 Piramida testów; metafora budownictwa

Wyobraź sobie budynek: solidne fundamenty (wiele małych testów) utrzymują ściany (mniej testów integracyjnych), a dach (nieliczne testy E2E) zamyka całość. Odwrotna piramida (dużo E2E, mało unit) = krucha budowla.

Dach: Testy E2E / UI (5%): wolne, kruche, kosztowne utrzymania
Ściany: Testy integracyjne (25%): Room + API + ViewModel razem
Fundamenty: Testy jednostkowe (70%): szybkie, izolowane, tanie

Google rekomenduje: 70% unit - 20% integration - 10% E2E.

5.1. Podział testów w projekcie Android

Typ testu	Lokalizacja / charakterystyka / narzędzia
`Unit tests (testy jednostkowe)`	src/test/ - JVM only, bez Androida. Milisekundy na test. Testuj: mapperów, ViewModel logiki, use caseów, util klas. Narzędzia: JUnit 5, MockK, Turbine, kotlinx-coroutines-test.
`Integration tests (testy integracyjne)`	src/test/ lub src/androidTest/ - Room InMemory + Fake Repository. Testuj: DAO queries, Repository flow, VM + Repository razem. Narzędzia: Room + JUnit 5.
`Instrumented tests (UI testy)`	src/androidTest/ - wymagają emulatora/urządzenia. Sekundy do minut. Testuj: Compose UI interakcje, nawigację, dostępność. Narzędzia: Compose Testing, Espresso, Hilt Testing.

5.2. Zależności testowe; co dodać do build.gradle.kts

Zależności testowe

```
// app/build.gradle.kts - sekcja dependencies
dependencies {
  // JUnit 5 - modern framework do testów jednostkowych
  testImplementation("org.junit.jupiter:junit-jupiter:5.10.2")
  testImplementation("org.junit.jupiter:junit-jupiter-params:5.10.2")  // @ParameterizedTest

  // MockK - mocking framework dla Kotlin (zamiast Mockito)
  testImplementation("io.mockk:mockk:1.13.12")

  // Turbine - testowanie Flow (od Cash App)
  testImplementation("app.cash.turbine:turbine:1.1.0")

  // Coroutines test support
  testImplementation("org.jetbrains.kotlinx:kotlinx-coroutines-test:1.8.1")

  // Compose Testing (Instrumented)
  androidTestImplementation("androidx.compose.ui:ui-test-junit4")
  debugImplementation("androidx.compose.ui:ui-test-manifest")  // Wymagany manifest dla testów
```

```
  // Hilt Testing
  androidTestImplementation(libs.hilt.testing)
  kspAndroidTest(libs.hilt.compiler)
}

// Który JUnit runner? Dla JUnit 5 potrzebny dodatkowy plugin:
tasks.withType<Test> { useJUnitPlatform() }  // W bloku android {} lub tasks
```

6. Testy jednostkowe - JUnit 5 + MockK

Test jednostkowy (unit test) testuje jedną klasę w izolacji. Wszystkie zależności zastępujemy obiektami pozorowanymi (mock). Dlatego Dependency Injection jest kluczowy dla testowalności!

Dlaczego MockK zamiast Mockito dla Kotlin?

Mockito powstał dla Javy i ma kilka problemów z Kotlinem:
1. Klasy final - w Kotlinie wszystkie klasy są domyślnie final. Mockito nie może ich mockować bez hacków.
2. Kotlin coroutines i suspend functions - Mockito nie wspiera natywnie, wymaga obejść.
3. Kotlin-specific API (companion object, extension functions, top-level functions) - Mockito nie obsługuje.

MockK został stworzony dla Kotlin od podstaw:
+ Mockuje klasy final natywnie (przez inline mocking).
+ Natywne wsparcie dla suspend functions (coEvery, coVerify).
+ Mocki companion objectów, object singletonów, extension functions.

6.1. Anatomia testu jednostkowego

PokemonMapperTest.kt - test mappera	Wyjaśnienie
`class PokemonMapperTest {`	Zwykła klasa Kotlin - bez @RunWith, bez Android
`  @Test`	@Test - adnotacja JUnit 5 (import org.junit.jupiter.api.Test)
`  fun `height in decimeters is converted to meters`() {`	Nazwa testu jako backtick string - czytelna dokumentacja
`    // GIVEN - przygotuj dane`	Sekcja Given: co mamy na wejściu (Arrange)
`    val dto = fakePokemonDetailDto(height = 7)`	Tworzymy DTO z wysokością 7 dm (= 0.7m)
`    // WHEN - wykonaj testowane działanie`	Sekcja When: co testujemy (Act)
`    val result = dto.toDomainModel()`	Wywołujemy mapper - to jest SUT (System Under Test)
`    // THEN - zweryfikuj wynik`	Sekcja Then: co oczekujemy (Assert)
`    assertThat(result.heightMeters).isEqualTo(0.7)`	Google Truth: czytelniejsze niż assertEquals
`  }`	
`  @Test`	
`  fun `hidden abilities are filtered out`() {`	
`    val dto = fakePokemonDetailDto(`	
`      abilities = listOf(`	
`        fakeAbility("static", isHidden = false),`	Widoczna zdolność - powinna pojawić się w wyniku
`        fakeAbility("lightning-rod", isHidden = true)`	Ukryta zdolność - powinna być odfiltrowana
`      )`	
`    )`	

PokemonMapperTest.kt - test mappera	Wyjaśnienie
`val result = dto.toDomainModel()`	
`assertThat(result.abilities).containsExactly("Static")`	Tylko 'static' (z wielką literą po capitalize)
`}`	
`}`	

6.2. Testowanie ViewModelu z MockK i Coroutines

PokemonListViewModelTest.kt	Wyjaśnienie
`@ExtendWith(`	@ExtendWith - JUnit 5, nie @RunWith (JUnit 4)
`CoroutinesTestExtension::class, // <- niestandardowe`	Instaluje TestCoroutineDispatcher jako Main dispatcher
`MockKExtension::class`	MockK extension automatycznie inicjalizuje @MockK pola
`)`	
`class PokemonListViewModelTest {`	
`@MockK`	@MockK tworzy mock repozytorium - nie prawdziwa implementacja!
`private lateinit var repository: PokemonRepository`	lateinit bo MockK inicjalizuje przez refleksję
`private lateinit var viewModel: PokemonListViewModel`	SUT - System Under Test
`@BeforeEach`	@BeforeEach - wywoływane PRZED każdym @Test (JUnit 5, nie @Before!)
`fun setUp() {`	
`viewModel = PokemonListViewModel(repository)`	Tworzymy VM przez constructor injection - dlatego DI jest ważne!
`}`	
`@Test`	
`fun `loading state is emitted first`() = runTest {`	runTest - z kotlinx-coroutines-test. Kontroluje czas coroutine.
`// GIVEN`	
`coEvery { repository.getPokemonList() } returns`	coEvery = mockK dla suspend fun. 'co' = coroutine
`flowOf(listOf(fakePokemon()))`	Możemy zwrócić Flow z daną wartością
`// WHEN + THEN z Turbine`	Turbine: .test() na Flow zbiera emisje
`viewModel.uiState.test {`	
`assertThat(awaitItem()).isInstanceOf(`	awaitItem() = poczekaj na następną emisję
`UiState.Loading::class.java)`	Pierwszy emitowany stan: Loading
`assertThat(awaitItem()).isInstanceOf(`	
`UiState.Success::class.java)`	Drugi stan po załadowaniu: Success
`cancelAndIgnoreRemainingEvents()`	Anuluj test Flow - oczyszczenie
`}`	
`}`	
`}`	

6.3. Wzorzec Given-When-Then i testy parametryczne

Dobry test składa się z trzech sekcji: Given (co mamy), When (co robimy), Then (co oczekujemy). Ten wzór - znany też jako Arrange-Act-Assert - sprawia, że testy są samodokumentujące.

ParameterizedTest dla mappera	Wyjaśnienie
`@ParameterizedTest`	@ParameterizedTest = jeden test, wiele danych wejściowych

ParameterizedTest dla mappera	Wyjaśnienie
`@CsvSource(`	@CsvSource: dane w formacie CSV wprost w adnotacji
`  "4, 0.4",  // 4 dm -> 0.4 m`	Każda linia to jeden przypadek testowy
`  "10, 1.0",  // 10 dm -> 1.0 m`	
`  "0, 0.0",  // 0 dm -> 0.0 m (krawędź)`	
`  "999, 99.9" // duża wartość`	
`)`	
`fun `height conversion from decimeters to meters`(`	Parametry testów jako argumenty funkcji
`  heightDm: Int, expectedM: Double`	JUnit 5 automatycznie wstrzykuje z @CsvSource
`) {`	
`  val dto = fakePokemonDetailDto(height = heightDm)`	
`  val result = dto.toDomainModel()`	
`  assertThat(result.heightMeters).isEqualTo(expectedM)`	
`}`	

7. Testy integracyjne - Room InMemory

Testy integracyjne sprawdzają, czy wiele komponentów działa ze sobą poprawnie. W kontekcie Room testujemy DAO queries z prawdziwą bazą danych - ale bazującą na pamięci RAM zamiast dysku.

🔍 Dlaczego Room InMemory do testów?

Testy na prawdziwej bazie SQLite miałyby kilka problemów:
1. STAN między testami - dane z poprzedniego testu widoczne w następnym. Testy muszą być izolowane.
2. SZYBKOŚĆ - operacje dyskowe są powolne. Testy powinny być szybkie.
3. Sprzątanie - po każdym teście trzeba ręcznie usuwać plik bazy.

Room InMemory Database:
+ Każde @BeforeEach tworzy NOWĄ bazę w pamięci. Zerowy stan początkowy.
+ @AfterEach zamknie bazę i pamięć zwalnia automatycznie.
+ Identyczne zachowanie jak SQLite, ale błyskawicznie szybka.
+ Nie wymaga emulatora - można uruchomić w src/test/ (JVM) z robolectric.

7.1. Test DAO z Room InMemory

PokemonDaoTest.kt	Wyjaśnienie
`@RunWith(AndroidJUnit4::class)`	AndroidJUnit4 - uruchamia test z prawdziwym Android runtime (na emulatorze)
`class PokemonDaoTest {`	Lub użyj @RunWith(RobolectricTestRunner) dla JVM
`  private lateinit var database: PokemonDatabase`	Baza testowa - tworzona na nowo przed każdym testem
`  private lateinit var dao: PokemonDao`	
`  @Before`	@Before (JUnit 4) lub @BeforeEach (JUnit 5)
`  fun setUp() {`	
`    database = Room.inMemoryDatabaseBuilder(`	KLUCZOWE: inMemoryDatabaseBuilder zamiast databaseBuilder
`      ApplicationProvider.getApplicationContext(),`	Kontekst aplikacji z testów Android
`      PokemonDatabase::class.java`	Ta sama klasa Database co w produkcji!
`    ).allowMainThreadQueries()  // Tylko w testach!`	Normalne Room blokuje Main thread. W testach to OK.
`    .build()`	

PokemonDaoTest.kt	Wyjaśnienie
`  dao = database.pokemonDao()`	DAO dostępny po stworzeniu bazy
`}`	
`@After`	@After = sprzątanie po każdym teście
`fun tearDown() { database.close() }`	Zamknij bazę - zwolnij pamięć
`@Test`	
`fun `upsert and retrieve pokemon`() = runTest {`	runTest dla suspend DAO queries
`  // GIVEN`	
`  val entity = fakePokemonEntity(id = 25, name = "pikachu")`	
`  // WHEN`	
`  dao.upsert(entity)`	Zapisz do bazy (w pamięci)
`  val result = dao.getPokemonById(25).first()`	.first() pobiera pierwszy element z Flow i kończy
`  // THEN`	
`  assertThat(result?.name).isEqualTo("pikachu")`	
`}`	
`}`	

7.2. Fake vs Mock - którego używać?

Podejście	Kiedy używać i dlaczego
`Mock (MockK / Mockito)`	Gdy chcesz ZWERYFIKOWAĆ interakcje (czy było wywołane? z jakimi argumentami?). Szybki w tworzeniu. Ryzyko: testy sprawdzają implementację, nie zachowanie - będą się psuć przy refaktoryzacji.
`Fake (ręczna implementacja)`	Gdy chcesz przetestować ZACHOWANIE z lekką implementacją. FakeRepository trzyma dane w pamięci. Testy są mniej kruche - nie zależą od kolejności wywołań. Przykład: FakePokemonRepository z MutableList.
`Room InMemory`	Gdy testujesz DAO i SQL queries. Prawdziwa logika bazodanowa, żerowa konfiguracja. Jedyne sensowne podejście dla testów bazy danych.
`Spy (MockK spyk)`	Gdy chcesz częściowo mockować prawdziwy obiekt. Rzadko potrzebny - często sygnalizuje problem z architekturą.

8. Testy instrumentowane: Compose UI Testing

Testy instrumentowane uruchamiają się na prawdziwym urządzeniu lub emulatorze i mogą testować zachowanie UI. Jetpack Compose dostarcza dedykowaną bibliotekę do testowania komponentów i ekranów.

8.1. Semantyka: jak Compose Testing widzi UI

Compose Testing nie operuje na View hierarchii (jak Espresso), lecz na drzewie semantycznym. Każdy Composable może ujawniać właściwości semantyczne: etykietę, rolę, stan (zaznaczony/niezaznaczony), wartość.

🔍 Drzewo semantyczne; czego nie widzi test

Compose Testing działa na DRZEWIE SEMANTYCZNYM, nie wizualnym layoutcie.
Drzewo semantyczne to 'opis dla narzędzi dostępności' - TalkBack i testy widzą to samo.

Konsekwencje dla testów:
+ Szukasz elementów przez contentDescription, text, role - nie przez ID widoków (jak w XML/Espresso).

+ onNodeWithText("Pikachu") znajdzie dowolny Text composable z tym tekstem.
+ onNodeWithContentDescription("sprite") znajdzie AsyncImage z tym contentDescription.

WAZNE: Jeśli composable nie ma ustawionej semantyki (Modifier.semantics), test go nie znajdzie! Dlatego contentDescription jest tak ważny - nie tylko dla TalkBack, ale i dla testów.

8.2. Anatomia testu Compose

PokemonListScreenTest.kt	Wyjaśnienie
`@HiltAndroidTest`	@HiltAndroidTest = Hilt obsługuje DI w tym teście instrumentowanym
`class PokemonListScreenTest {`	
`  @get:Rule(order = 0)`	Order 0: HiltRule MUSI być pierwsza (inicjalizuje Hilt przed regułą Compose)
`  val hiltRule = HiltAndroidRule(this)`	HiltAndroidRule inicjalizuje Hilt dla każdego testu
`  @get:Rule(order = 1)`	Order 1 - po inicjalizacji Hilt
`  val composeTestRule = createAndroidComposeRule<MainActivity>()`	createAndroidComposeRule startuje Activity z Hilt
`  @Before`	
`  fun setUp() { hiltRule.inject() }`	hiltRule.inject() - wstrzykuje @Inject pola w klasie testowej
`  @Test`	
`  fun `pokemon list shows items after loading`() {`	
`    composeTestRule.apply {`	
`// Poczekaj aż Loading zniknie (maks 5 sekund)`	
`      waitUntilDoesNotExist(`	waitUntil - poczekaj na warunek asynchronicznie
`        hasTestTag("loading_indicator"),`	TestTag - pewniejsze niż szukanie po tekście
`        timeoutMillis = 5_000`	
`      )`	
`// Sprawdź, czy lista zawiera min. jeden element`	
`      onNodeWithTag("pokemon_list")`	Znajdź element po testTag
`        .assertIsDisplayed()`	Zweryfikuj że jest widoczny
`      onAllNodesWithTag("pokemon_card")`	onAllNodes = wiele elementów
`        .assertCountAtLeast(1)`	Co najmniej 1 karta Pokemon
`    }`	
`  }`	
`}`	

8.3. Selektory i akcje - API Compose Testing

Selector / Akcja	Opis i przykład
`onNodeWithText("text")`	Znajdź node z danym tekstem. Domyślnie: substring match. Dodaj substring=false dla dokładnego.
`onNodeWithContentDescription("opis")`	Znajdź po contentDescription. Niezbędne dla obrazów i ikon.
`onNodeWithTag("tag")`	Znajdź po Modifier.testTag(). Najbardziej stabilny selektor - nie zmienia się przy tłumaczeniach.
`onAllNodesWithTag("tag")`	Zwraca SemanticsNodeInteractionCollection. Użyj [index] lub assertCountAtLeast().
`.performClick()`	Akcja: kliknięcie na element.
`.performTextInput("text")`	Akcja: wpisanie tekstu w TextField.
`.performScrollTo()`	Akcja: przewinięcie do elementu (w LazyColumn).

Selector / Akcja	Opis i przykład
`.assertIsDisplayed()`	Asercja: element widoczny na ekranie.
`.assertTextEquals("text")`	Asercja: tekst elementu jest dokładnie 'text'.
`.assertIsEnabled() / Disabled()`	Asercja: czy przycisk jest aktywny/nieaktywny.

8.4 TestTag - klucz do stabilnych testów

Dodawanie testTagów w Compose	Wyjaśnienie
`// W komponencie produkcyjnym:`	
`LazyVerticalGrid(`	
`  modifier = Modifier.testTag("pokemon_list")`	testTag identyfikuje element dla testów
`) {`	
`  items(pokemons) { pokemon ->`	
`    PokemonCard(`	
`      pokemon = pokemon,`	
`      modifier = Modifier.testTag("pokemon_card")`	Każda karta ma ten sam tag - użyj onAllNodes
`    )`	
`  }`	
`}`	
`CircularProgressIndicator(`	
`  modifier = Modifier.testTag("loading_indicator")`	Tag na spinnerze - czekamy aż zniknie
`)`	

⚠ testTag nie powinien trafić do release APK w produkcji

Modifier.testTag() nie zwalnia pamięci, ale dodaje semantykę. Dostępną potencjalnie póxniej dla reverse engineeringu. W produkcie korporacyjnym rozważ użycie semanticsRole lub warunkowego testTag:

modifier = if (BuildConfig.DEBUG) Modifier.testTag("pokemon_list") else Modifier

W projektach akademickich i większości komercyjnych jest to akceptowalne w kodzie produkcyjnym. Google własne Sample Apps (Jetpack Compose Samples) używa testTag bez warunkowania.

9. Hilt w testach: podmienianie zależności

Kluczową zaletą Hilt jest możliwość podmieniania prawdziwych implementacji na testowe (fake/stub) bez zmiany kodu produkcyjnego. Hilt oferuje dwa mechanizmy: @TestInstallIn i @UninstallModules.

9.1. Podmienianie modułów w testach

FakeRepositoryModule.kt - moduł testowy	Wyjaśnienie
`@TestInstallIn(`	@TestInstallIn zamienia moduł produkcyjny na testowy
`  components = [SingletonComponent::class],`	W którym komponencie? Singleton - tak jak oryginał
`  replaces = [RepositoryModule::class]`	Który moduł produkcyjny zastępujemy?
`)`	
`@Module`	
`abstract class FakeRepositoryModule {`	

FakeRepositoryModule.kt - moduł testowy	Wyjaśnienie
`@Binds @Singleton`	
`abstract fun bindRepository(`	Zamiast PokemonRepositoryImpl, Hilt wstrzyknie FakeRepository
`fake: FakePokemonRepository`	FakePokemonRepository musi mieć @Inject constructor
`): PokemonRepository`	Interfejs jest ten sam - ViewModel nie wie o zamianie!
`}`	
`// FakePokemonRepository.kt:`	
`class FakePokemonRepository @Inject constructor(`	
`// Brak sieci! Lista w pamięci jako 'baza'`	
`) : PokemonRepository {`	Implementuje ten sam interfejs
`private val pokemons = MutableList<Pokemon>(`	
`mutableListOf(fakePokemon(25, "pikachu"))`	Predefiniowane dane testowe
`)`	
`override fun getPokemonList() =`	
`flowOf(pokemons.toList())`	Zwraca Flow z danymi testowymi
`}`	

🔍 Jak Hilt scala moduły w testach?

W normalnej aplikacji: Hilt ładuje NetworkModule + RepositoryModule - prawdziwy Retrofit + Room.

W teście z @TestInstallIn:
1. Hilt widzi że FakeRepositoryModule ma replaces = [RepositoryModule::class].
2. RepositoryModule jest IGNOROWANY - jego @Provides/@Binds nie są rejestrowane.
3. FakeRepositoryModule jest załadowany zamiast niego.
4. Reszta modułów (NetworkModule) nadal działa - tylko podmieniony moduł jest zastąpiony.

@UninstallModules (alternatywa, per-test): Adnotacja na klasie testowej. Pozwala podmienić moduł tylko dla jednego testu. @TestInstallIn = globalnie dla wszystkich testów w tym sourceset.

10. Testowanie Coroutines: TestDispatcher i Turbine

Coroutines są asynchroniczne - test musi umieć kontrolować czas ich wykonania. Bez tego testy będą niestabilne (flaky) - czasem zaliczone, czasem nie, bez zmiany kodu.

10.1. Problem z asynchronicznością w testach

TestDispatcher - kontrola czasu coroutine

```
// Bez TestDispatcher - test BŁĘDY:
class BrokenViewModelTest {
  @Test
  fun `data loads correctly`() {
    viewModel.loadData()          // Uruchamia coroutine NA PRAWDZIWYM IO dispatcher
    // Coroutine jest ASYNCHRONICZNA - wynik nie jest jeszcze gotowy!
    assertThat(viewModel.uiState.value)
      .isInstanceOf(UiState.Success::class.java)  // FAIL: nadal Loading
  }
}

// Rozwiązanie: StandardTestDispatcher + advanceUntilIdle()
class FixedViewModelTest {
  private val testDispatcher = StandardTestDispatcher()  // Kontrolowany scheduler
```

```
  @BeforeEach fun setUp() {
    Dispatchers.setMain(testDispatcher)  // Podmień Main dispatcher na testowy
  }

  @AfterEach fun tearDown() {
    Dispatchers.resetMain()              // Przywróć po teście
  }

  @Test
  fun `data loads correctly`() = runTest(testDispatcher) {
    viewModel.loadData()           // Coroutine ZAPLANOWANA, ale nie uruchomiona
    advanceUntilIdle()             // Uruchom WSZYSTKIE oczekujące coroutines
    assertThat(viewModel.uiState.value)
      .isInstanceOf(UiState.Success::class.java)  // OK: coroutine zakończona
  }
}
```

10.2. Turbine: testowanie Flow

Testowanie Flow bez Turbine wymaga boilerplate kodu: uruchom kolektor w tle, zbieraj do listy, anuluj, porównaj. Turbine upraszcza to do kilku czytelnych wywołań.

Turbine - asercje na Flow	Wyjaśnienie
`viewModel.uiState.test {`	.test { } - Turbine blokuje i zbiera emisje Flow
`  // awaitItem() czeka na następną emisję (suspend - poczeka!)`	
`  val first = awaitItem()`	Pierwsza emisja: oczekujemy Loading
`  assertThat(first).isInstanceOf(UiState.Loading::class.java)`	
`  val second = awaitItem()`	Druga emisja: po załadowaniu danych
`  assertThat(second).isInstanceOf(UiState.Success::class.java)`	
`  assertThat((second as UiState.Success).data)`	Cast do Success, sprawdź dane
`    .hasSize(20)`	20 Pokemonów na pierwszej stronie
`  // Anuluj, nie czekamy na kolejne emisje (Flow może być nieskończony)`	
`  cancelAndIgnoreRemainingEvents()`	Bezpieczne zakończenie testu Flow
`}`	
`// Alternatywnie dla błędów:`	
`viewModel.uiState.test {`	
`  awaitItem()  // Loading`	
`  val errorState = awaitItem()`	
`  assertThat(errorState).isInstanceOf(UiState.Error::class.java)`	
`  assertThat((errorState as UiState.Error).message)`	
`    .contains("Brak połączenia")`	
`  cancelAndIgnoreRemainingEvents()`	
`}`	

Test Doubles - słownik terminów

Dummy: obiekt przekazany ale niewywoływany (wypełniacz parametru).

Stub: zwraca predefiniowane wartości, nie weryfikuje wywołań. Prostszy niż Mock.

Fake: lekka implementacja (np. in-memory repository). Zachowuje logikę ale bez zewnętrznych zależności.
Mock: weryfikuje interakcje (czy było wywołane? ile razy? z jakimi args?). Może też stubbować.
Spy: opakowuje prawdziwy obiekt, pozwala obserwować wywołania i częściowo mockować.

11. Strategia testowania, czyli co testować w projekcie

Wiedza o technikach to jedno - ważniejsza jest odpowiedź na pytanie: które części kodu WARTO testować? Nie wszystko wymaga testów. Koncentruj się tam, gdzie błędy są kosztowne lub trudne do zauważenia.

11.1. Priorytety testowania

Co testować (wysoki priorytet)	Co można pominąć lub testować ręcznie
`Mappery (DTO → Domain) - przeliczenia jednostek, nullable, capitalize, filtrowanie`	Proste gettery/settery - brak logiki = brak testów
`ViewModel logika - kolejność stanów (Loading→Success→Error), paginacja, retry`	Activity / Fragment boilerplate - Hiltów@AndroidEntryPoint
`DAO queries - upsert, delete, flow emissions, migracje Room`	Compose composableów bez logiki - czyste UI które tylko wyświetla dane
`safeApiCall - obsługa każdego typu wyjątku (IOException, HttpException, Gson)`	Konfiguracja Hilt modułów - kompilator sprawdza poprawność
`Użytkowe funkcje biznesowe - walidacja formularzy, obliczenia, parsowanie dat`	Trzecie party biblioteki - nie testujesz Retrofit, Coil, Room

11.2. Sekwencja pisania testów TDD

#	Test Driven Development - Red, Green, Refactor
1	RED: Napisz test, który FAILUJE. Opisz co chcesz osiągnąć (Given-When-Then). Uruchom - test musi być czerwony.
2	GREEN: Napisz MINIMALNY kod produkcyjny, który sprawi, że test przechodzi. Nie optymalizuj. Nie przewiduj. Napisz dosłownie minimum.
3	REFACTOR: Ulepsz kod produkcyjny (i test!) bez zmiany zachowania. Testy są siatką bezpieczeństwa - jeśli po refaktoryzacji są zielone, refaktoryzacja była bezpieczna.
4	POWTARZAJ: Każda nowa funkcjonalność = nowy test = nowy cykl Red-Green-Refactor.

12. Zadania do wykonania

Zadania podzielone są na dwie części: integracja Hilt do istniejących projektów (TaskApp + PokeApp) oraz pisanie testów weryfikujących logikę biznesową i interfejs użytkownika.

Zadanie 1 (25 pkt) - Migracja TaskApp do Hilt

1.1. Dodaj wtyczkę Hilt i KSP do build.gradle.kts. Dodaj zależności (hilt-android, hilt-compiler, hilt-navigation-compose).
1.2. Stwórz klasę TaskApp z adnotacją @HiltAndroidApp i zarejestruj ją w AndroidManifest.xml.
1.3. Napisz DatabaseModule (@Module, @InstallIn(SingletonComponent::class)) dostarczający TaskDatabase i TaskDao przez @Provides @Singleton.
1.4. Zmień TaskRepository na klasę z @Inject constructor. Dostosuj TaskViewModel do @HiltViewModel @Inject constructor.
1.5. Dodaj @AndroidEntryPoint do MainActivity. Zastąp viewModel() na hiltViewModel() w TaskListScreen.
WERYFIKACJA: Aplikacja kompiluje się i działa identycznie jak przed migracją. Brak singletonów ręcznych.

Zadanie 2 (20 pkt) - Testy jednostkowe mappera i safeApiCall

2.1. Napisz min. 5 testów jednostkowych dla PokemonMapper (JUnit 5, bez Androida, bez MockK):

a) przeliczenie decymetrów na metry,
b) przeliczenie hektogramów na kilogramy,
c) capitalize nazwy,
d) filtrowanie ukrytych zdolności,
e) fallback dla null baseExperience.

2.2. Napisz testy @ParameterizedTest dla przeliczeń jednostek z co najmniej 4 przypadkami.

2.3. Napisz min. 3 testy dla safeApiCall() używając MockK (coEvery):

a) sukces (Result.Success),
b) HttpException → Result.Error z kodem,
c) IOException → Result.Error.

WERYFIKACJA: ./gradlew test kończy się bez błędów. Pokrycie kodu mappera ≥ 80% (sprawdzone w Android Studio).

Zadanie 3 (30 pkt) - Testy integracyjne i ViewModelu

3.1. Napisz min. 3 testy DAO (Room InMemory, AndroidJUnit4 lub Robolectric):

a) upsert + pobranie, b) usunięcie + weryfikacja braku, c) Flow emituje aktualizację po upsert.

3.2. Stwórz FakePokemonRepository implementujący PokemonRepository (dane w MutableList, Flow z SharedFlow).

3.3. Napisz min. 4 testy PokemonListViewModel (runTest, Turbine, MockK):

a) kolejność stanów Loading→Success,
b) błąd sieci → UiState.Error z komunikatem,
c) retry po błędzie → ponowna próba (verify coVerify),
d) paginacja - loadMore zwraca więcej elementów.

WERYFIKACJA: ./gradlew test kończy się bez błędów. Wszystkie 10+ testów zielone.

Zadanie 4 (25 pkt) - Testy instrumentowane Compose UI

4.1. Dodaj testTag do kluczowych komponentów: pokemon_list, pokemon_card, loading_indicator, error_message, retry_button.

4.2. Skonfiguruj FakeRepositoryModule (@TestInstallIn) podmieniający PokemonRepository na FakePokemonRepository.

4.3. Napisz min. 3 testy instrumentowane (createAndroidComposeRule, @HiltAndroidTest):

a) lista wyświetla karty po załadowaniu,
b) kliknięcie karty otwiera ekran szczegółów (nawigacja),
c) FakeRepository zwracający błąd → widoczny ErrorScreen z przyciskiem 'Spróbuj ponownie'.

4.4. Uruchom testy na emulatorze: ./gradlew connectedAndroidTest. Wszystkie zielone.

WERYFIKACJA: Demonstracja uruchomienia testów na emulatorze prowadzącemu. Raport HTML z wynikami.

13. Kryteria oceniania

13.1. Punktacja zadań

Zadanie	Punkty	Co weryfikuje prowadzący
Zad. 1: Hilt w TaskApp	25 pkt	Kompilacja bez błędów. Brak ręcznych singletonów. @HiltViewModel + hiltViewModel(). DatabaseModule poprawny.
Zad. 2: Testy mappera + safeApiCall	20 pkt	./gradlew test zielony. Min. 5 testów JUnit 5. @ParameterizedTest. Testy safeApiCall z MockK (coEvery).
Zad. 3: Testy integracyjne + ViewModel	30 pkt	Room InMemory DAO test. FakeRepository. Turbine dla Flow. Testy VM z verify/coVerify.
Zad. 4: Testy Compose UI	25 pkt	FakeRepositoryModule @TestInstallIn. Testy instrumentowane na emulatorze. Raport HTML.
RAZEM	100 pkt	

13.2. Skala ocen

Ocena	Punkty	Wymagania
5.0	90–100 pkt	Wszystkie 4 zadania. Min. 15 testów. Testy instrumentowane na emulatorze. Pokrycie ≥80%.
4.5	80–89 pkt	Zadania 1–3 kompletne. Testy instrumentowane - min. 1 test UI przechodzi na emulatorze.
4.0	70–79 pkt	Zadania 1–3. Hilt działa. Min. 8 testów jednostkowych i integracyjnych.
3.5	60–69 pkt	Zadanie 1 kompletne + zadanie 2 (min. 5 testów jednostkowych). Brak testów VM lub DAO.
3.0	50–59 pkt	Hilt skonfigurowany (@HiltAndroidApp, @HiltViewModel). Min. 3 testy jednostkowe mappera.
2.0	0–49 pkt	Projekt nie kompiluje się z Hilt, lub brak jakichkolwiek testów.

14. Najczęstsze błędy i ich rozwiązania

Hilt i testy mają swoje specyficzne pułapki. Poniższa tabela zbiera błędy, które studenci napotykają najczęściej. Sprawdź ją zanim zaczniesz szukać w Internecie.

Komunikat błędu / objaw	Przyczyna i rozwiązanie
`[Hilt] ... is not an @AndroidEntryPoint`	Klasa Activity/Fragment/ViewModel nie ma adnotacji. Dodaj @AndroidEntryPoint do MainActivity i @HiltViewModel do ViewModela.
`[Hilt] ... cannot be provided without @Inject or @Provides`	Hilt nie wie jak stworzyć tej klasy. Dodaj @Inject constructor() do klasy LUB napisz @Provides w Module LUB @Binds dla interfejsu.
`Expected single matching node but found 0`	Compose Testing nie znalazł elementu. Sprawdź: testTag pasuje? Element na ekranie? Dodaj Modifier.testTag(). Użyj printToLog() do debug.
`CancellationException w teście ViewModela`	Nie użyto runTest lub brakuje advanceUntilIdle(). Uruchom logikę VM w runTest { } i wywołaj advanceUntilIdle() przed asercją.
`MockKException: no answer found for ...`	Nie skonfigurowałeś coEvery dla wywołanej metody. Dodaj coEvery { repo.method() } returns ... PRZED wywołaniem kodu testowanego.
`Cannot access database on the main thread`	Brakuje allowMainThreadQueries() w Room InMemory dla testów. LUB: Nie używasz runTest dla suspend queries.
`@TestInstallIn not replacing module`	FakeModule musi być w src/androidTest/ (dla testów instrumentowanych) LUB src/test/ (dla JVM testów). Zły katalog = moduł nie jest widoczny.
`Hilt rule must be first in @get:Rule order`	HiltAndroidRule musi mieć order=0, ComposeTestRule order=1. Hilt inicjalizuje się przed innymi regułami.
`ViewModel is not initialized (Hilt)`	Brakuje @HiltAndroidApp na klasie Application LUB klasa Application nie jest zarejestrowana w Manifeście android:name=".MyApp".

Instrukcja Laboratoryjna Nr 5: Hilt, Testy jednostkowe i instrumentacyjne
Programowanie Aplikacji Mobilnych | Katedra Informatyki i Automatyki
MobileHub
Następne ćwiczenie: Lab 6 - WorkManager, powiadomienia Push i DataStore Preferences